



POLITECNICO
MILANO 1863

Formal Analysis of Search-and-Rescue Scenarios

Homework Project

Course Name

Formal Methods For Concurrent and Real-Time Systems
A.Y. 2023/2024

Instructors

Prof. [REDACTED]

Dr. [REDACTED]

Authors

Alberto [REDACTED]

Lorenzo [REDACTED]

Contents

1 Abstract	3
2 Model Description	3
2.1 Layout	3
2.2 Communication	3
2.3 Termination	4
2.4 Templates	4
2.4.1 Initializer	4
2.4.2 Civilian	4
2.4.3 First-Responder	6
2.4.4 Drone	7
3 Properties	8
3.1 Scenarios	8
3.1.1 Scenario 1: Urban High-Rise Fire	9
3.1.2 Scenario 2: Large-Scale Natural Disaster	9
3.1.3 Scenario 3: Industrial Complex Emergency	9
3.2 Stochastic Scenario Versions	9
3.3 Verification Results (Non-Stochastic)	9
3.3.1 Scenario 1 Results	10
3.3.2 Scenario 2 Results	10
3.3.3 Scenario 3 Results	10
3.4 Verification Results (Stochastic)	11
3.4.1 Scenario 1 Results	11
3.4.2 Scenario 2 and 3 Results	11
4 Conclusions	12

1 Abstract

This document presents an UPPAAL model for search-and-rescue scenarios, the design choices behind it, meaningful properties and their formal verification, with the final objective of highlighting its strengths and weaknesses. The modelled problem is the search and rescue of civilians during an emergency, carried out by trained professional assisted by autonomous drones. In this document it is assumed that the reader has full access to the described UPPAAL model and the problem description.

2 Model Description

From here onwards the word *entity* refers to a civilian, first-responder or drone (and all of their subcategories), while *object* refers to fire and exit. Each entity is uniquely identified among other entities of the same type by an id. While going through the different aspects and sections of the model, the corresponding design decisions will also be explicitly highlighted and explained.

2.1 Layout

Map and Positions The modelling of the emergency area is achieved by a 2D array, called for simplicity map. While the position of each entity is specified in its respective 1D array, where the particular entity is identified by its index and each element of the array is a pair of map coordinates.

Each map cell contains only one integer value, and each cell value type has a different meaning. The most meaningful and critical decisions to explain are two. The first one is the absence of a drone-value in the map, deemed superfluous and a complicating factor due to the following considerations: drones can fly over other entities and objects, and the adopted drone's moving policy (explained in section 2.4.4). The second point is that the states of *SAFE* and *CASUALTY*, while different in concept, are written in the map as the same: *FREE*. This last choice was done because when civilians are saved or dead, they are no longer part of the scenario, so the cell that they once occupied should be now free and their position in the respective array removed.

General Movement and Detection Entities move following their respective moving policy, updating the map 2D array and its 1D position array. The following *simplifying assumption* was made: entities can only move in the four cardinal directions (and remain in the same position for first-responder and civilian). For this reason distances between cells are calculated with the Manhattan distance. This choice was made to simplify the moving policy without loss of generality, since the model can be easily extended with eight-direction movement without impacting the overall model and logic. On the other hand, entities can detect other entities and objects in all directions. This choice was made to simulate the realistic behaviour of entities, which could not be simplified like for the moving policy. This decision places greater emphasis on the detection policies.

2.2 Communication

As it will be clearly explained in future sections, entities have the need to communicate in a specific ways.

Channels A given entity need to synchronize with a specific entity over a channel, to achieve this, binary correspondence channel arrays were utilized, where the specific target entity for that channel is identified by its id. Each channel is clearly named according to the communication that it represents.

Message Passing In addition to synchronization, entities also need to pass information between each other and to solve this problem a communication mechanism was implemented. The model simulates a postman carrying letters to different recipients, managing all of the message passing and cleaning the inboxes whenever necessary. At the core of the system there are messages (single or double, depending on the number of values contained), then each entity has its own inbox containing a single message, finally all of inboxes are grouped into a single array (identified by the entity id who owns them) contained in a global structure (communication purpose specific).

2.3 Termination

In a real-life scenario a rescue operation cannot continue indefinitely, drones will run out of battery, first-responders will get tired and civilians who are not rescued in time become missing. To model this important aspect, every entity has its own "tiredness meter" implemented by an integer in their respective indexed arrays. The single counter measures how much an entity is "tired" and not how much energy it has left (for ease of implementation without clarity loss). Whenever, an entity does a meaningful amount of work its counter will increase until it reaches its "max tiredness threshold" after which the entity will not be able to do any more work. This aspect will not be further discussed in the templates, if not to by pointing out interesting and meaningful details. Each entity category has its own single threshold, this could be expanded into a new parameter for each entity.

2.4 Templates

The model is based on the following 4 templates:

- *Initializer*: helping template to initialize the scene and the various used data structures.
- *Civilian*: person in need of rescue inside the emergency area, who can receive instructions by a drone, become missing, casualty or saved.
- *First-Responder*: trained professional rescuing civilians inside the emergency area.
- *Drone*: autonomous flying entity, which has the objective of instructing civilians in the best way possible to maximizing rescue rate.

2.4.1 Initializer

Simple automaton to correctly populate the map with entities and to initialize all the global data structures. Following this initialization all the other automata are signalled to start.

2.4.2 Civilian

A civilian can be:

- *CIVILIAN*: default state, also referred to as survivor.
- *SAFE*: saved civilian.
- *VICTIM*: civilian near a fire.
- *CASUALTY*: deceased civilian.
- *IN_ASSISTANCE*: civilian who is being assisted by a responder.
- *ZERO_RESPONDER*: civilian who was instructed by a drone to help a victim.
- *IN_CONTACT*: civilian who was instructed by a drone to contact a first-responder.
- *EXHAUSTED_CIVILIAN*: civilian who is too tired to move.

All these different subcategories were used to explicitly model the different conceptual states in which a civilian can be in and their evolution during a scenario. In addition, this clear distinction enables the model to utilize more precise, easier and cleaner logic in order to avoid many problems related to miss-communication, miss-detection and multi-communication conflicts.

A civilian is characterized by the following parameters:

- *civilian_id*
- *tv*: time in seconds before a victim dies, becoming a casualty.
- *tzr*: time in seconds needed for a zero-responder to rescue a victim.

Behaviour What follows is the general behaviour and the most important aspects of the civilian template, not all transitions, states and details will be discussed.

A civilian can check its surroundings up to 1-cell distance and it has no knowledge about the layout of the emergency area (*simplifying assumption*).

1. A civilian starts off by going through three initial committed states in which it is checked if it is *safe*, *victim* or *exhausted civilian*, if none of those are true it ends up in an *Idle* state. The committed nature of these first states is fundamental, since no other entity can move apart for civilians and this assures that the model behaves as intended. Because at each civilian's step, before any other entity can do anything, that civilian will correctly update its state, not misleading any other entity by being in a wrong state. In addition, this also works perfectly for the initialization phase of civilians in the map.
2. When a civilian becomes a *victim*, it will become a *casualty* after t_v time. However, if in this period of time it is assisted by a responder, it can be saved. If the civilian is *in assistance* and becomes a *casualty* this event will be synchronized with the responder. What happens when a civilian becomes *safe* or *casualty* inside the map has been already described in section [2.1](#).
3. When a civilian becomes an *exhausted civilian*, it will be unable to move any further. If a first-responder sees it, the civilian can still be rescued and saved. However, if all drones and first-responders have run out of energy and the scene has ended (calculated by an over estimate of the maximum amount of time that it might take), the rescue is considered completed and the civilian will become a missing civilian. There is no value type for this last status, since it is not needed in the map.
4. From *Idle* state, a *civilian* can be contacted by a drone or it can move randomly (*civilian moving policy*), since it has no knowledge about the emergency area. A civilian will stay idle for a finite amount of time. As a simplifying assumption it was chosen the value 2 for all civilians, since they might need some time to decide where to move next (not trained professionals). This aspect could be further expanded into a new parameter for each civilian.
5. When it is contacted by a drone, it can be instructed to become a *zero-responder* or an *in contact* civilian. The committed states are fundamental to create atomic transitions.

- (a) In the *zero-responder* scenario, the civilian receives a message from the drone indicating which *victim* requires assistance. The civilian then acts as a responder, assisting both itself and the identified victim. If the *victim* becomes a *casualty* before the rescue is completed, the *zero-responder* will still complete its own rescue, simulating the fact that it now knows where the exit is and the ability to reach it independently.
- (b) In the *in-contact* scenario, the civilian receives a message from the drone that identifies which *first-responder* to contact and which *victim* needs assistance. The civilian simulates the move towards the *first-responder*, which takes time based on the *distance* between them. Then the civilian waits until the *first-responder* is available and then communicates (via an urgent channel to ensure synchronization without delays) the identity of the *victim* in need. After communication, the civilian will receive assistance from the *first-responder*.

However, if during the approach to the *first-responder* or while waiting for it to become available, the *victim* no longer requires assistance (whether because they have been rescued, have become a casualty, or are currently receiving help), the civilian will refrain from contacting the *first-responder*. This prevents wasting the valuable time of the *first-responder*. Moreover, the first-responder does not assist the contacting survivor even if it is waiting near itself, because the contacting survivor is not in need of assistance in that moment and the rescue had not yet started. So, the first-responder should prioritize rescuing other civilians who are in need (in a real-life scenario the first-responder could simply inform the contacting survivor where the nearest exit is).

Finally, a civilian cannot wait indefinitely for the *first-responder*. A waiting time of 5 was chosen for all civilians, to be balanced in regard to t_{fr} and n_v values. This parameter could be further tested to assess its impact on the model's performance.

Stochastic Version To account for the stochastic features the civilian template has been expanded:

- New civilian parameter *plisten*: probability expressed by an integer [0, 100], that a civilian when given an instruction by a drone follows it.
- New probabilistic transition: when a civilian is contacted by a drone, now there is a branch point. A civilian will follow the instruction (accepting the communication) only with probability *plisten*, while it will ignore it with probability $100 - plisten$. This simulates the unpredictability of untrained civilians under stress.
- New channel *civilian_noncompliance_msg* and global communication structure *global_drone_comm_civilian*: these two additions (and the necessary ones that follow) are necessary to make the drone aware that the civilian did not follow the given instruction and to act accordingly.

2.4.3 First-Responder

A first-responder can be:

- *FIRST_RESPONDER*: default first-responder.
- *CONTACTED_FIRST_RESPONDER*: first-responder who is being contacted by a survivor.
- *FATIGUED_FIRST_RESPONDER*: first-responder who is too tired to continue the rescue.

This differentiation was made to ensure that: no two drones could detect the same first-responder and no drone could detect a *fatigued first-responder* (according to the decision policy of the drone described in section 2.4.4), and to give a higher priority to the assisting of a contacting survivor and the corresponding victim, since this action can save two civilians instead of one.

A first-responder is characterized by the following parameters:

- *first_responder_id*
- *tfr*: time in seconds needed for a first-responder to rescue a victim.

Behaviour What follows is the general behaviour and the most important aspects of the first-responder template, not all transitions, states and details will be discussed.

A first-responder can check its surroundings up to 1-cell distance and it has knowledge about the objects layout in the emergency area (*conceptually* needed to assist others and not die).

1. A first-responder begins in an *Idle* state. From this state, and following the previously discussed action priorities, the first-responder will, if not fatigued, scan its surroundings to locate a *victim* or *exhausted civilian* in need of assistance. If multiple civilians are detected, the first-responder will prioritize saving the one with the highest *tv* (*first-responder detection policy*). This approach is based on the real-life need for first-responders to make decisions about who to assist first. Since the *tfr* is fixed (each rescue takes the same amount of time), it makes sense for the first-responder to assist the civilian with the greatest chance of survival. The model could be extended to accommodate variable *tfr* values depending on different factors (e.g., different types of rescues requiring different *tfrs*) or to incorporate different priorities for different types of civilians (e.g., children, men, women, elderly, etc.).

A first-responder remains idle for a finite amount of time to determine its next course of action. As a simplifying assumption, a value of 1 was chosen for all first-responders, considering that they are trained professionals whose primary objective is to assist civilians; thus, they must make decisions quickly. This aspect could be expanded into a unique parameter for each first-responder.

- (a) If a first-responder detects a *victim* in need of help, it will initiate the rescue, completing it within the *tfr* time frame. If the *victim* dies during the rescue, becoming a *casualty*, the rescue will be aborted.

- (b) If a first-responder does not detect a *victim*, it will move randomly (*first-responder moving policy*), as it has no prior knowledge about civilians' positions.

A first-responder will take a finite amount of time to execute its moving policy. As a simplifying assumption, a value of 1 was chosen for all first-responders, based on their need to act quickly. This aspect could be expanded into a distinct parameter for each first-responder.

The introduction of a committed state after the detection of a *victim* is crucial to ensure atomic actions and prevent conflicts, as no other entity (e.g., other first-responders or drones) should be able to detect and act upon that *victim*. This models a plausible rescue coordination mechanism among entities during an emergency.

Furthermore, distinguishing between this committed state and the *Wandering* state is essential to avoid potential deadlocks, which are also prevented by the discussed moving policy.

No interactions occur when a first-responder is fatigued, as it cannot be detected by drones. To achieve this, the function *fEnergyUsed* was employed during victim detection, even though this does not fully reflect real-life scenarios where the first-responder would typically become fatigued after performing an action. In this model, the first-responder is already fatigued when making the final rescue or movement.

2. When a first-responder is contacted (*contacted first-responder*), it will, after completing its previous rescue or movement, assist both the *contacting survivor* and the identified *victim*. This assistance will take $tfr + \text{distance between itself and the victim}$ time. However, if the *victim* becomes a *casualty* during this time, the first-responder will continue to rescue the *contacting survivor*, as aborting the rescue entirely would result in the loss of all progress made, wasting precious time.

2.4.4 Drone

Drones have been modelled as if they would be all deployed from one side of the map (without loss of generality, from the top) with the aim of maximizing the covered area, so they are meaningfully spaced apart. The idea behind is that when the rescue team arrives at the emergency location they can deploy the fleet of drones from one side of the area. This could be a practical way of deploying them and not only this design choice increases their rescue capabilities, but it also prevents most communication conflicts between them.

As discussed in section [2.1](#), drones are not represented in the map.

A drone is characterized by the following parameters:

- *drone_id*
- *nv*: range in cells in which a drone can detect other entities.

Behaviour What follows is the general behaviour and the most important aspects of the drone template, not all transitions, states and details will be discussed.

1. A drone starts off in an *Idle* state. From here it can scan its surrounding area to detect meaningful entities, if it is not energy drained. A drone will take a finite amount of time to process information. As a simplifying assumption it was chosen the value 1 for all drones, since the internal processing of a drone is rather fast. This aspect could be further expanded into a new parameter for each drone.

The *drone detection policy* is as follows: it will scan within a square radius of *nv* cells its surroundings, looking for *survivors*, *victims* and *first-responders*; if there is more than one entity for a specific category, it will choose the closest one. The following assumption was made: a drone does not have the capability to choose the victim with the highest possibility to live (like a first-responder can). The drone can only detect *victims* and not *exhausted civilians*, since it cannot distinguish between them and a normal *civilians*. In addition, if the model would be extended to support variable first-responder's *tfr*, the drone could have the capability to use this additional information to make better decisions; the same consideration applies with the addition of civilian's priorities.

- (a) If a drone detects a *survivor* and *victim* pair, it will start the communication with the survivor and then give instructions according to the *drone decision policy*. If the drone during its scanning also detected a *first-responder*, then it will instruct the survivor to become an *in contact* civilian, giving it the additional information needed about which first-responder to contact and which victim needs assistance. On the contrary, if there is no near *first-responder*, the drone instructs the survivor to become a *zero-responder*. This design choice ensures the utilization of first-responders' expertise to help two civilians, instead of one, whenever possible, while also offering the option of helping the victim in need by providing instructions to a fellow civilian to assist it.
- (b) If a drone does not detect a *survivor* and *victim* pair, it will move following the *drone moving policy*. When drones are deployed into the map, they will start from the top edge and move downwards in a straight line. Then to change direction and come back, the drones do not fly until the limit of the map, but they take into consideration their *nvs* in order to cover more efficiently the map. This refinement is achievable by using simple sensors, that should already be present on the drones. This moving policy has been chosen for its simplicity and practicality in real-life scenarios, because this drone fleet configuration is much easier to control, it covers the most area without using more complex policies and it helps avoiding conflicts between drones, which would create additional confusions in an already difficult and chaotic situation.

The following are some additional considerations worth pointing out:

- When a drone decides to instruct a survivor to contact a first-responder, the drone itself sets the first-responder value type to *contacted first-responder*. This is done to ensure no conflicts among drones, and could be achieved in real-life by drones communicating which first-responder has been assigned (recognizing the first-responder by a chip tag). This could also be extended to support quasi-real-time monitoring of first-responders.
- The drone's committed states are fundamental to ensure atomic actions and avoid conflicts, since no other entity should be able to detect the detected *survivor*, *victim* or *first-responder* (e.g. other drones or first-responders) and act upon that detection. Moreover, entities should not move and change state (e.g. civilians or first-responders) while the detection and decision is taking place. The *Patrolling* state has been also marked as committed, even though not strictly mandatory. This was done to convey the idea of the atomic action that a drone makes during its internal processing, since it follows a predetermined path.

Stochastic Version To account for the stochastic features the drone template has been expanded:

- New drone parameter *pfail*: probability expressed by an integer [0, 100], that the detection of entities done by the scanning of a drone does not work.
- New probabilistic transition: when a drone detects a *survivor* and *victim* pair, now there is a branch point. A drone will start the communication only with probability $100 - pfail$, while it will ignore the detection with probability *pfail*. This simulates the defect affecting drones' vision sensors.

3 Properties

3.1 Scenarios

The followings are the three designed scenarios to evaluate the above described model: Urban High-Rise Fire, Large-Scale Natural Disaster, and Industrial Complex Emergency. The following table summarizes their key parameters:

There are a finite number of possible states combinations, due to the terminating conditions discussed in section 2.3 and this number depends on the "max tiredness threshold"s selected. For all the scenarios these thresholds were set to 5, to have a good number of states, but not too many. And already with such small numbers the total number of possibilities was very high, highlighting the rapid growth of complexity.

These scenarios assess system performance in different conditions, such as dense urban vs. widespread disaster areas, limited vs. advanced drone capabilities, and varying civilian vulnerability.

Parameter	Scenario 1	Scenario 2	Scenario 3
Environment	Urban High-Rise	Natural Disaster	Industrial Complex
Map Size	8x10	20x14	18x12
N.Drones (visibility)	2 ($n_v = 2$)	2 ($n_v = 3$)	4 ($n_v = 1$)
N.First-Responders (rescue time)	2 ($t_{fr} = 2 - 3$)	5 ($t_{fr} = 8$)	5 ($t_{fr} = 4$)
N.Civilians (time to live, zero-responder rescue time)	3 ($t_v = 2 - 11$, $t_{zr} = 1 - 2$)	11 ($t_v = 15 - 25$, $t_{zr} = 6$)	10 ($t_v = 15 - 25$, $t_{zr} = 5$)

Table 1: Summary of Emergency Response Scenarios

3.1.1 Scenario 1: Urban High-Rise Fire

This scenario tests efficiency in a compact, high-stakes environment. The compact map simulates a dense urban environment with advanced drones, with for example thermal imaging capabilities. Limited responders reflect initial teams, while varied civilians' time to live test prioritization in time sensitive situations.

3.1.2 Scenario 2: Large-Scale Natural Disaster

This scenario tests handling of complex, large-scale situations with limited but advanced rescue entities. The large map represents a widespread disaster area. High drones' visibility tests advanced drones allocation over a vast area. Similar high rescue times reflect environmental challenges, while high civilians' time to live simulates a well prepared population.

3.1.3 Scenario 3: Industrial Complex Emergency

This scenario tests operations in a hazardous environment with low-visibility drones. The medium-sized map simulates varied industrial terrain. Low-visibility drones reflect operational challenges in potentially hazardous conditions. High rescue times account for the complex environment, while varied civilians' time to live represent different risk levels across the industrial complex.

3.2 Stochastic Scenario Versions

The stochastic scenarios are equal to the non-stochastic ones above described. The only additions are the new required parameters for the drone and the civilian templates. To simulate a wide range of realistic possibilities the values of *pfails* and *plistsens* across the three different scenarios are well balanced. A rescue level drone should not fail more than 10% of the times, while it was made the optimistic assumption that a civilian, even under stress, would listen with probability not below 75%.

To enable the probabilistic model checking, it was necessary to convert all channels into broadcast ones.

3.3 Verification Results (Non-Stochastic)

This section discusses the results of two queries on three systems to assess the efficiency and reliability of civilian rescue operations under various constraints. The key variables were $N\%$ (percentage of civilians to be rescued) and T_{scs} (time constraint for rescue).

The queries in question are:

- $E\langle \rangle \text{ n_safe } \geq \text{ n_percentage } \ \&\& \ \text{global_time} < \text{t_scs} \ \&\& \ \text{initializer.Initialized}$
Check if at least one path allows rescuing $N\%$ of civilians within T_{scs} .
- $A\langle \rangle \text{ n_safe } \geq \text{ n_percentage } \ \&\& \ \text{global_time} < \text{t_scs} \ \&\& \ \text{initializer.Initialized}$
Check if all paths lead to rescuing at least $N\%$ of civilians within T_{scs} .

These queries evaluate the possibility ($E\langle \rangle$) and inevitability ($A\langle \rangle$) of rescuing a certain percentage of civilians within a time frame.

For the tables 'V' indicates a successful query, while ' ∞ ' means that the query was running for a long time, meaning the result could be successful or not (but is very likely it will eventually turn not successful).

3.3.1 Scenario 1 Results

The second query always returned false, indicating no guaranteed rescue within the time constraints. The first query gave more varied results, summarized below:

$T_{scs} \backslash N\%$	25%	50%	75%	100%
8	F	F	F	F
9	V	F	F	F
10	V	F	F	F
11	V	F	F	F
12	V	V	V	F
13	V	V	V	F
14	V	V	V	F
15	V	V	V	V

Table 2: Scenario 1 - Query 1 Results

These results show that significant rescues are possible given enough time, highlighting the importance of quick response and efficient entities allocation early in rescue operations.

3.3.2 Scenario 2 Results

For the second scenario, our analysis revealed distinct patterns in the rescue operation's dynamics. The outcomes for the first query are summarized in the following table:

$T_{scs} \backslash N\%$	1	2	25%	50%	75%	100%
5	V	V	∞	∞	∞	∞
10	V	V	∞	∞	∞	∞
100	V	V	∞	∞	∞	∞

Table 3: Scenario 2 - Query 1 Results

For the second query, the results show that it returns true if and only if the number of people to be saved is 1 and this happens regardless of the time constraints, due to the fact that there is a civilian near an exit.

These findings suggest that scenario 2 has a more constrained rescue capability compared to scenario 1. The system provides a consistent level of performance for small-scale rescues but faces significant challenges in scaling up to larger percentages of the civilian population.

3.3.3 Scenario 3 Results

Scenario 3 exhibits characteristics similar to scenario 2, with the difference that query 2 returns always false, since there is no civilian near exit.

Key observations for scenario 3:

1. The scenario maintains the possibility of saving up to 2 civilians across all time frames.
2. As with scenario 2, increasing the time constraint does not improve rescue potential for larger groups.
3. The false result for query 2 suggests that even saving a single civilian is not guaranteed under all circumstances.
4. The scenario can never guarantee for rescuing 25% or more of the civilians.

Scenario 3 presents a more challenging rescue environment compared to scenario 1 and 2. While it retains the potential for medium-scale rescues, it lacks any guaranteed successful outcomes, indicating a higher level of uncertainty in rescue operations.

3.4 Verification Results (Stochastic)

In addition to the deterministic queries, it was decided to conduct a stochastic analysis on the three scenarios using the following query:

```
Pr[<=t_scs] (<> n_safe >= n_percentage && initializer.Initialized)
```

Returns the probability that in less than or equal time to T_{scs} , the condition of safety is respected.

3.4.1 Scenario 1 Results

Key findings include:

- Time Range: The study was conducted for time intervals ranging from 8 to 15, mirroring the non-stochastic model.
- Consistent Low Probability for 25% Rescue:
 - When time ≥ 9 , the probability of saving 25% of the population remains constant at 15%.
- Time Threshold for Higher Rescue Percentages:
 - Rescuing 50% or more of the population becomes feasible when time ≥ 12 , albeit with a very low probability.
 - At time ≥ 12 , the probability of saving 50% or more people reaches a minimum viable level of 5%.
- Extreme Rescue Scenarios:
 - Even at the maximum studied time ($t = 15$), the probability of saving 100% of the population remains extremely low.
- Observations:
 - The study reveals a clear relationship between time and rescue probability, with longer time intervals generally allowing for higher chances of successful rescues.
 - However, even with increased time, the probabilities of saving larger percentages of the population remain relatively low.
 - There appears to be a significant threshold at $t = 12$, after which more ambitious rescue operations become possible, albeit still challenging.
 - The findings suggest that while extended time improves rescue capabilities, other factors likely play crucial roles in determining the overall success of large-scale rescue operations.
 - These results underscore the complexity of rescue scenarios and highlight the need for efficient strategies that can maximize the number of lives saved within given time constraints.

3.4.2 Scenario 2 and 3 Results

For both Scenario 2 and Scenario 3, similar patterns emerged in the stochastic analysis:

- Probability of rescuing 25% of civilians:
 - At $t = 5$: 5% success rate
 - At $t = 30$: 15% success rate
 - At $t = 50$: 20% success rate
- Probability of rescuing 50% of civilians:
 - At $t = 5$: 0% success rate
 - At $t = 15$: 0% success rate

- At $t = 50$: 0% success rate

The stochastic analysis of Scenarios 2 and 3 provides valuable insights into the challenges and limitations of large-scale emergency response operations:

1. Time-dependent rescue probability: As time increases, the probability of rescuing a quarter of the civilians also increases, but with diminishing returns. This suggests that while extended operation time is beneficial, other factors become limiting as time progresses.
2. Difficulty in large-scale rescues: The consistently low (often zero) probability of rescuing 50% or more of the civilians, even with extended time, indicates significant challenges in achieving large-scale rescues in these scenarios.
3. Scalability issues: The stark contrast between the probabilities of rescuing 25% versus 50% of civilians highlights potential scalability problems in the rescue operations.
4. Environmental complexity: The similarity in results between Scenarios 2 and 3 suggests that both the large-scale natural disaster and the industrial complex emergency present comparable levels of difficulty for rescue operations.
5. Resource limitations: The low success rates, even with extended time, may indicate that the available resources (drones and first responders) are insufficient for the scale of these emergencies.
6. Importance of early response: The slight increase in success probability for 25% rescue between $t=5$ and $t=30$ emphasizes the critical nature of the initial response period.
7. Long-term challenges: The minimal improvement in success rates beyond $t=50$ suggests that prolonged operations face diminishing returns, possibly due to factors like resource depletion or environmental changes.

These results underscore the complexity of large-scale emergency responses and highlight the need for strategies that can effectively allocate limited resources, prioritize early interventions, and potentially increase the scale of response capabilities to improve outcomes in challenging scenarios.

4 Conclusions

Our analysis of search-and-rescue scenarios using UPPAAL has yielded several important insights:

1. The effectiveness of rescue strategies varies significantly across different emergency situations, underscoring the necessity for adaptable approaches.
2. Time constraints emerge as a critical factor in rescue operations. Extended timeframes generally correlate with higher success rates, particularly when attempting to rescue larger groups of civilians.
3. The stochastic analysis reveals a stark contrast between single-civilian and large-scale rescues. While individual rescues demonstrate high reliability, operations involving a larger percentage of civilians (e.g., 25%) are considerably more time-sensitive and less predictable.
4. Notably, none of the modelled systems achieved a consistent 100% rescue rate, highlighting the inherent complexities and uncertainties in emergency response scenarios.
5. The model illustrates the intricate interconnection of all the various entities, objects and parameters.
6. Potential areas for improvement include optimising drones allocation, enhancing the policies of drones and responders (as already proposed), and developing more versatile systems capable of adapting to diverse emergency contexts.

In conclusion, this study demonstrates the significant value of formal modelling techniques in analysing and enhancing emergency response systems. While providing valuable insights into current model capabilities, it also shows promising ways for future research and improvements in this critical field.